

CodeAlpha Internship Program

Task 3: Handwritten Character Recognition*Project Report*

A Convolutional Neural Network for Recognizing Handwritten Digits, Uppercase and Lowercase Characters (EMNIST ByClass)

Field	Details
Student Name	Hassan Bahar
Institution	University of Management and Technology
Internship	CodeAlpha
Task	Task 3 — Handwritten Character Recognition
Submission Date	10/10/2026

Table of Contents

Table of Contents	2
List of Figures	4
List of Tables	4
2. Abstract	5
3. Introduction	6
4. Problem Statement	7
5. Objectives.....	8
6. Dataset Description.....	9
6.1 Dataset Statistics.....	9
6.2 Class Distribution	9
6.3 Suitability of EMNIST ByClass.....	10
7. Data Preprocessing	11
8. Data Augmentation.....	12
9. CNN Model Architecture.....	13
10. Model Compilation and Training	14
11. Training Results.....	15
12. Model Evaluation	16
13. Classification Report	17
13.1 Strong-Performing Classes	17
13.2 Weak-Performing Classes	17
14. Confusion Matrix.....	18
15. Results and Discussion	19
15.1 Overall Performance	19
15.2 Strengths	19
15.3 Weak Classes and Class Imbalance	19
15.4 Effect of Preprocessing and Augmentation	19
15.5 Generalization to Unseen Test Data	19
16. Prediction / Inference System.....	20
17. User Interface	21
18. Applications.....	22
19. Limitations.....	23
20. Future Improvements	24
21. Conclusion.....	25

22. Technologies and Tools.....	26
23. References	27
24. Appendix	28
Appendix A — Full Per-Class Classification Report (Test Set)	28
Appendix B — Key Code Snippets.....	30
Appendix C — Additional Screenshots and Prediction Examples.....	30

(Right-click the table above and choose "Update Field" in Microsoft Word to populate page numbers.)

List of Figures

Figure 1: EMNIST Dataset Sample Characters

Figure 2: Data Augmentation Examples (Original vs Augmented)

Figure 3: CNN Architecture / Model Summary

Figure 4: Training and Validation Accuracy Curve

Figure 5: Training and Validation Loss Curve

Figure 6: Confusion Matrix

Figure 7: Final Handwritten Character Recognition UI (Gradio Interface)

List of Tables

Table 1: Class Distribution of the EMNIST ByClass Dataset

Table 2: CNN Model Architecture Summary

Table 3: Training and Validation Performance per Epoch

Table 4: Overall Test Set Evaluation Metrics

Table 5: Per-Class Classification Report (Appendix)

2. Abstract

This project implements a handwritten character recognition system capable of classifying digits, uppercase letters, and lowercase letters using a Convolutional Neural Network (CNN). The system is built on the EMNIST ByClass dataset, which contains 62 character classes (10 digits, 26 uppercase letters, and 26 lowercase letters) derived from the NIST handwriting database. A total of 697,932 labelled training images and 116,323 labelled test images, each a 28x28 grayscale image, were used. Of the training images, 10% (69,794 images) were set aside as a stratified validation split, leaving 628,138 images for model training.

The raw pixel data was normalized to the $[0, 1]$ range and reshaped to include a single grayscale channel before being passed to the network. A compact CNN was designed consisting of two convolutional blocks (32 and 64 filters respectively, each followed by max pooling), a flattening layer, a 128-unit fully connected layer with dropout regularization (rate 0.30), and a final 62-way softmax output layer, totalling 428,350 trainable parameters. The model was compiled with the Adam optimizer (learning rate 0.001) and sparse categorical cross-entropy loss, and trained for up to 15 epochs with a batch size of 256, using early stopping, learning-rate reduction on plateau, and model checkpointing to preserve the best-performing weights.

On the held-out test set of 116,323 images, the final CNN achieved a test loss of 0.3537 and a test accuracy of approximately 87.04%. The corresponding per-class evaluation yielded a macro-average F1-score of 0.7418 and a weighted-average F1-score of 0.8565, reflecting strong performance on digits and many letters, alongside weaker performance on a subset of visually ambiguous lowercase letters. Data augmentation layers (random rotation, translation, and zoom) were implemented and their effect visualized on sample images, although the final training run used the non-augmented normalized data directly. The trained model was subsequently integrated into an interactive Gradio-based prediction interface that accepts an uploaded handwritten character image, automatically preprocesses it into EMNIST-style input, and returns the predicted character together with its confidence score and top-5 alternative predictions.

3. Introduction

Handwritten character recognition is a long-standing problem in computer vision and pattern recognition that involves teaching a machine to interpret handwritten input — whether digits, letters, or symbols — and convert it into a machine-readable form. It sits at the core of many practical Optical Character Recognition (OCR) systems and has applications ranging from postal automation and bank cheque processing to digitizing historical documents and enabling handwriting-based user interfaces.

Unlike printed text, handwritten characters vary significantly from person to person in stroke thickness, slant, size, and shape, making the recognition task considerably more challenging than recognizing machine-printed fonts. Deep learning, and Convolutional Neural Networks (CNNs) in particular, has become the dominant approach for this problem because CNNs can automatically learn hierarchical, spatially-invariant features directly from pixel data without the need for hand-crafted feature engineering.

This project applies a CNN-based approach to the EMNIST ByClass dataset, an extension of the classic MNIST digit dataset that additionally includes uppercase and lowercase English letters, to build an end-to-end handwritten character recognition system with an accompanying interactive prediction interface.

4. Problem Statement

Automatically recognizing isolated handwritten characters from images is a challenging multi-class classification problem for several reasons. First, the same character can be written in many visually distinct ways depending on the writer's style. Second, several characters are visually similar to one another — for example, the digit '0' and the letter 'O', the digit '1' and the lowercase letter 'l', or a lowercase letter and its uppercase counterpart such as 'c' and 'C', 's' and 'S', or 'o' and 'O' — which increases ambiguity when the classification space spans all 62 classes simultaneously. Third, real-world datasets of handwritten characters are typically class-imbalanced, since certain letters occur far less frequently than digits or common letters in natural writing samples.

The goal of this project is to build a CNN-based classifier that can take a 28x28 grayscale image of an isolated handwritten character as input and correctly predict which of the 62 possible classes (0-9, A-Z, a-z) it belongs to, and to expose this capability through a usable prediction interface that can classify new, user-supplied handwriting samples.

5. Objectives

The specific objectives of this project are to:

- Build a complete handwritten character recognition system covering data acquisition, preprocessing, model training, evaluation, and inference.
- Use the EMNIST ByClass dataset, covering all 62 character classes (digits, uppercase letters, and lowercase letters).
- Preprocess the handwritten character images (normalization and reshaping) into a form suitable for a CNN.
- Design, compile, and train a Convolutional Neural Network for 62-class character classification.
- Evaluate the trained model on a held-out test set using accuracy, loss, precision, recall, and F1-score.
- Analyze class-wise performance to identify which characters the model recognizes well and which it struggles with.
- Build a user-friendly prediction interface that allows a user to upload a handwritten character image and receive a predicted character, confidence score, and top alternative predictions.

6. Dataset Description

The project uses the EMNIST ByClass dataset, one of the split configurations of the Extended MNIST (EMNIST) dataset. EMNIST ByClass extends the original MNIST handwritten digit dataset by including handwritten uppercase and lowercase English letters, giving a total of 62 distinct classes: the digits 0-9, the uppercase letters A-Z, and the lowercase letters a-z. In the notebook, the dataset was obtained as gzip-compressed IDX-format image and label files (train/test images and labels, plus a class-to-character mapping file) downloaded from a Hugging Face dataset mirror of EMNIST ByClass, and parsed manually using the IDX binary format rather than through the `tensorflow_datasets` library.

6.1 Dataset Statistics

The exact sample counts and image dimensions confirmed directly from the notebook output are as follows:

- Number of classes: 62 (10 digit classes, 26 uppercase letter classes, 26 lowercase letter classes)
- Original downloaded training images: 697,932 (shape 28 x 28)
- Original downloaded test images: 116,323 (shape 28 x 28)
- Training split after stratified validation split (90% / 10%): 628,138 images
- Validation split (10% of the original training set, stratified, `random_state = 42`): 69,794 images
- Test set (held out, unchanged): 116,323 images
- Image dimensions: 28 x 28 pixels, single-channel grayscale (reshaped to 28 x 28 x 1 before being fed to the CNN)
- Dataset source: Hugging Face dataset mirror "Royce30ne/emnist-byclass" (IDX uubyte.gz format), following the standard EMNIST orientation-correction transform (transpose) applied during loading

6.2 Class Distribution

The dataset is naturally class-imbalanced: digit classes (0-9) each contain roughly 31,000-38,000 training samples, whereas letter classes range from as few as 1,896 samples (lowercase 'j') to as many as 24,983 samples (uppercase 'O'). This imbalance, confirmed directly from the notebook's class distribution output on the training set, is discussed further in Section 15 (Results and Discussion). The full class-wise training sample counts are presented in Table 1.

Class	Count	Class	Count	Class	Count	Class	Count
0	34,585	G	2,517	W	4,695	m	2,645
1	38,374	H	3,152	X	2,771	n	11,418
2	34,203	I	11,946	Y	4,743	o	2,749
3	35,143	J	3,762	Z	2,701	p	2,448
4	33,535	K	2,468	a	10,033	q	2,994
5	31,416	L	5,076	b	5,159	r	14,105
6	34,232	M	9,002	c	2,854	s	2,699

Class	Count	Class	Count	Class	Count	Class	Count
7	35,754	N	8,237	d	10,177	t	18,262
8	33,946	O	24,983	e	24,631	u	2,830
9	33,847	P	8,347	f	2,561	v	2,910
A	6,407	Q	2,605	g	3,687	w	2,697
B	3,878	R	5,073	h	8,738	x	2,822
C	10,094	S	20,764	i	2,725	y	2,365
D	4,562	T	9,820	j	1,896	z	2,725
E	4,934	U	12,602	k	2,491		
F	9,182	V	4,637	l	15,318		

Table 1: Class distribution of the EMNIST ByClass training set (per-character sample counts, as printed in the notebook).

6.3 Suitability of EMNIST ByClass

EMNIST ByClass is well suited to this task because it directly extends the widely used MNIST benchmark to the full alphanumeric character space needed for general handwritten character recognition (not just digits), while retaining the same 28x28 grayscale image format. This allows a CNN architecture proven on MNIST-style digit recognition to be extended, with minimal changes, to the substantially harder 62-class problem covering digits and both letter cases.

7. Data Preprocessing

The following preprocessing steps were actually applied to the EMNIST ByClass data in the notebook, in order:

- Image loading from the raw IDX uubyte.gz files using a custom parser (struct/gzip), followed by an orientation-correction transpose applied to all images to match the conventional upright EMNIST character orientation.
- Stratified train/validation split: 10% of the original training set (69,794 images) was set aside as a validation set using scikit-learn's `train_test_split` with `test_size = 0.10`, `random_state = 42`, and `stratify = y_train`, preserving the per-class proportions between the training and validation subsets.
- Pixel normalization: all image arrays (training, validation, and test) were cast to float32 and divided by 255.0, rescaling pixel intensities from the original [0, 255] range to [0, 1]. This was verified in the notebook, where the resulting minimum and maximum pixel values were confirmed to be 0.0 and 1.0 respectively.
- Channel-dimension reshaping: a trailing channel axis was added to every image array (`X_train`, `X_val`, `X_test`), converting each array's shape from (N, 28, 28) to (N, 28, 28, 1), the input format required by the Conv2D layers of the CNN.

No additional preprocessing steps (such as binarization, resizing beyond 28x28, or histogram equalization) were present in the notebook for the training pipeline. [Not available in the notebook: any preprocessing steps beyond normalization and reshaping for the training pipeline.]

8. Data Augmentation

A Keras Sequential data augmentation pipeline was defined in the notebook, consisting of three randomized image transformation layers:

- RandomRotation — factor 0.05, fill_mode = "constant", fill_value = 0.0
- RandomTranslation — height_factor 0.08, width_factor 0.08, fill_mode = "constant", fill_value = 0.0
- RandomZoom — height_factor (-0.08, 0.08), width_factor (-0.08, 0.08), fill_mode = "constant", fill_value = 0.0

These layers were applied to a sample training image to visually demonstrate their effect, producing nine randomly augmented versions of the same character alongside the original for comparison. Data augmentation of this kind is useful for handwritten character recognition because it exposes the model to controlled variations in rotation, position, and scale that mimic the natural variability of human handwriting, which can help the model generalize better to unseen handwriting styles and reduce overfitting.

Note: The augmentation pipeline (data_augmentation) was defined and its output visualized on a sample image, but it was not incorporated as a layer inside the final CNN model (cnn_model_1), and the model.fit() call used to train the final model was run directly on the normalized, non-augmented X_train/y_train arrays. The reported training and evaluation results therefore reflect a model trained without augmentation actually applied during training.

[INSERT SCREENSHOT HERE — Figure 2: Original vs Augmented Handwritten Character Images]

Figure 2: Original vs Augmented Handwritten Character Images

9. CNN Model Architecture

The final working model (referred to in the notebook as `cnn_model_1`) is a sequential Convolutional Neural Network built with Keras, consisting of two convolutional blocks followed by a fully connected classification head. The exact architecture and parameter counts, taken directly from the `model.summary()` output in the notebook, are shown in Table 2.

Layer	Type	Output Shape	Parameters
Input	Input	(None, 28, 28, 1)	0
conv2d	Conv2D (32 filters, 3x3, ReLU, same padding)	(None, 28, 28, 32)	320
max_pooling2d	MaxPooling2D (2x2)	(None, 14, 14, 32)	0
conv2d_1	Conv2D (64 filters, 3x3, ReLU, same padding)	(None, 14, 14, 64)	18,496
max_pooling2d_1	MaxPooling2D (2x2)	(None, 7, 7, 64)	0
flatten	Flatten	(None, 3136)	0
dense	Dense (128 units, ReLU)	(None, 128)	401,536
dropout	Dropout (rate = 0.30)	(None, 128)	0
dense_1	Dense (62 units, Softmax)	(None, 62)	7,998

Table 2: CNN model architecture summary (`cnn_model_1`).

Total parameters: 428,350 (1.63 MB), all of which are trainable; there are no non-trainable parameters.

[INSERT SCREENSHOT HERE — Figure 3: CNN Architecture / Model Summary]

Figure 3: CNN Architecture / Model Summary

10. Model Compilation and Training

The final CNN model was compiled and trained with the following exact configuration, as specified in the notebook:

- Optimizer: Adam, with an initial learning rate of 0.001
- Loss function: `sparse_categorical_crossentropy`
- Evaluation metric: accuracy
- Batch size: 256
- Maximum number of epochs: 15
- Early stopping: `monitor = val_loss`, `patience = 3`, `restore_best_weights = True`
- Learning-rate reduction: `ReduceLROnPlateau`, `monitor = val_loss`, `factor = 0.5`, `patience = 2`, `min_lr = 1e-6`
- Model checkpointing: `ModelCheckpoint` saving to `best_cnn_model_1.keras`, `monitor = val_accuracy`, `save_best_only = True`, `mode = "max"`
- Class weighting: not applied to the final CNN — class weights were computed separately in the notebook (see Section 15) but were only used for the experimental EfficientNet-based pipeline, not for `cnn_model_1`.

Training ran for the full 15 scheduled epochs. Early stopping (monitoring validation loss with a patience of 3) triggered at the end of epoch 15 and restored the model's weights to the end of epoch 12, the point of lowest validation loss within the patience window. The learning-rate scheduler reduced the learning rate twice during training: from $1.0e-3$ to $5.0e-4$ after epoch 10, and from $5.0e-4$ to $2.5e-4$ after epoch 13.

11. Training Results

Table 3 lists the training and validation accuracy and loss recorded at the end of each epoch, taken directly from the notebook's training log.

Epoch	Train Acc.	Train Loss	Val Acc.	Val Loss	Learning Rate
1	78.73%	0.6774	84.80%	0.4300	1.0e-3
2	83.86%	0.4655	85.70%	0.3958	1.0e-3
3	84.82%	0.4300	86.07%	0.3869	1.0e-3
4	85.34%	0.4104	86.34%	0.3754	1.0e-3
5	85.72%	0.3965	86.52%	0.3727	1.0e-3
6	85.99%	0.3858	86.49%	0.3703	1.0e-3
7	86.21%	0.3772	86.58%	0.3673	1.0e-3
8	86.36%	0.3699	86.67%	0.3645	1.0e-3
9	86.59%	0.3628	86.59%	0.3659	1.0e-3
10	86.76%	0.3573	86.62%	0.3648	1.0e-3
11	87.32%	0.3383	86.99%	0.3607	5.0e-4
12	87.45%	0.3331	86.90%	0.3606	5.0e-4
13	87.57%	0.3289	86.90%	0.3628	5.0e-4
14	87.89%	0.3188	87.01%	0.3621	2.5e-4
15	88.00%	0.3150	87.00%	0.3635	2.5e-4

Table 3: Training and validation performance per epoch (cnn_model_1).

The best validation accuracy achieved during training was 87.01%, reached at epoch 14, as printed directly in the notebook ("Best Validation Accuracy: 87.01% / Best Epoch: 14"). Note that this differs slightly from the epoch (12) at which early stopping restored the model's weights, since early stopping tracks validation loss while the separately computed "best epoch" statistic tracks validation accuracy.

[INSERT SCREENSHOT HERE — Figure 4: Training and Validation Accuracy]

Figure 4: Training and Validation Accuracy

[INSERT SCREENSHOT HERE — Figure 5: Training and Validation Loss]

Figure 5: Training and Validation Loss

12. Model Evaluation

The final CNN model was evaluated on the held-out EMNIST ByClass test set of 116,323 images. The exact results reported by `model.evaluate()` and the subsequent `classification_report()` call in the notebook are summarized in Table 4.

Metric	Value
Test Loss	0.3537
Test Accuracy	87.04%
Macro Average Precision	79.81%
Macro Average Recall	73.93%
Macro Average F1-score	74.18%
Weighted Average Precision	86.22%
Weighted Average Recall (= Accuracy)	87.04%
Weighted Average F1-score	85.65%

Table 4: Overall test set evaluation metrics.

The gap between the macro-average F1-score (74.18%) and the weighted-average F1-score (85.65%) indicates that overall accuracy is driven largely by the strong performance on the numerically dominant and generally easier digit classes, while a subset of less frequent, harder-to-distinguish letter classes pull the unweighted (macro) average down considerably. This is examined in detail in Section 13.

13. Classification Report

The full 62-class precision/recall/F1-score classification report generated on the test set is reproduced in Appendix A (Table 5). The discussion below highlights the strongest and weakest performing classes, using only the actual figures from that report.

13.1 Strong-Performing Classes

Several classes achieved F1-scores above 0.95, including the digits '2' (0.9685), '3' (0.9932), '4' (0.9737), '6' (0.9781), '7' (0.9907), '8' (0.9813), uppercase letters 'A' (0.9607) and 'N' (0.9576), and lowercase letters 'd' (0.9824), 'e' (0.9810), and 'r' (0.9671). These classes are generally either visually distinctive digits or letters with few close look-alikes elsewhere in the 62-class label space.

13.2 Weak-Performing Classes

The weakest classes, by F1-score, were concentrated among a specific group of lowercase letters: 'o' (0.0127), 's' (0.0136), 'c' (0.0352), 'f' (0.0465), 'u' (0.1103), 'm' (0.2057), and 'l' (0.3292). Among digits and uppercase letters, the digit '0' (0.7543), digit '1' (0.7823), and uppercase 'I' (0.5557), 'O' (0.5933), and 'V' (0.6065) also scored comparatively low. In most of these weak cases the recall is markedly lower than the precision (for example, 's' has a precision of 0.7500 but a recall of only 0.0069), indicating that the model rarely predicts these classes at all rather than confusing them evenly across many other classes — consistent with the model favoring the visually similar uppercase counterpart ('S', 'C', 'O', 'U', 'F', 'M', 'L') for a large share of these lowercase examples.

This pattern is consistent with a well-known characteristic of the EMNIST ByClass label space: several lowercase letters are near-identical in shape to their uppercase counterparts when handwritten (e.g., c/C, o/O, s/S, u/U, v/V, w/W, x/X, z/Z, and to a lesser extent i/I, j/J, k/K, m/M, p/P), which makes the 62-way classification problem intrinsically harder for these specific letters than the raw digit or distinctive-shape classes.

14. Confusion Matrix

A confusion matrix visualizes how often the model's predictions for each true class matched or were mistaken for each other class, with rows typically representing the true label and columns the predicted label; a strong diagonal indicates accurate classification, while off-diagonal concentrations reveal systematic confusions between specific class pairs.

Note: [Not available in the notebook]: the uploaded notebook does not contain code that computes or plots a confusion matrix. Based on the classification report figures in Section 13, the confusion pattern most likely to appear if a confusion matrix were generated would concentrate around lowercase/uppercase letter pairs with near-identical shapes (e.g., o -> O, s -> S, c -> C, u -> U) and between visually similar digit/letter pairs (e.g., 0 -> O, 1 -> l/l), but this inference is not a substitute for an actual computed confusion matrix.

[INSERT SCREENSHOT HERE — Figure 6: Confusion Matrix]

Figure 6: Confusion Matrix

15. Results and Discussion

15.1 Overall Performance

The final CNN model reached 87.04% test accuracy with a test loss of 0.3537 across all 62 classes, a solid result for a relatively lightweight architecture (428,350 parameters) trained without data augmentation actually applied during training and without class-weighting for class imbalance.

15.2 Strengths

The model performs very well on digits and on letters with distinctive shapes, several of which reached F1-scores above 0.97 (e.g., digit '3' at 0.9932, digit '7' at 0.9907, lowercase 'd' at 0.9824). This suggests the two-convolution-block CNN architecture is well suited to learning the relatively simple, low-variability stroke patterns of digits and clearly shaped letters.

15.3 Weak Classes and Class Imbalance

Performance is markedly weaker on a specific set of lowercase letters that closely resemble their uppercase counterparts ('c', 'f', 'o', 's', 'u', 'm', 'l', among others — see Section 13.2), several of which also have comparatively small training sample counts (Table 1), for example 'j' (1,896 samples), 'k' (2,491), 'o' (2,749), and 'p' (2,448). The combination of visual ambiguity with the corresponding uppercase class and lower per-class training volume plausibly compounds the difficulty for these classes, though the notebook does not include a controlled experiment isolating the two factors.

15.4 Effect of Preprocessing and Augmentation

Normalization to $[0, 1]$ and reshaping to include an explicit channel dimension were the two preprocessing steps actually applied before training, and both are standard requirements for feeding image data into a Keras Conv2D-based network. As noted in Section 8, the augmentation pipeline was implemented and visualized but was not applied during the actual training run for the final CNN, so its potential benefit to generalization was not captured in the reported results.

15.5 Generalization to Unseen Test Data

The final test accuracy (87.04%) is close to the best validation accuracy recorded during training (87.01% at epoch 14), indicating that the model generalized consistently from the validation split to the fully held-out test set, without evidence of severe overfitting given the available metrics.

16. Prediction / Inference System

The trained CNN model (saved to `/content/final_emnist_cnn.keras`) is loaded and used to power an interactive prediction system. The end-to-end inference workflow implemented in the notebook is as follows:

- 1. User uploads a handwritten character image through the interface.
- 2. The image is converted to grayscale, and the code automatically detects whether the image has a light or dark background (based on mean pixel intensity) and inverts it if necessary so the character is represented as bright strokes on a dark background, matching the EMNIST convention.
- 3. A pixel-intensity threshold (value 30) is applied to locate the character region, and a bounding box is computed around the non-background pixels; if no character is detected, the system reports "Not Detected".
- 4. The character is cropped to its bounding box, placed onto a square canvas (padded symmetrically to preserve aspect ratio), resized to 24x24 pixels using Lanczos resampling, and pasted into a blank 28x28 canvas with a 2-pixel border — reproducing the general framing convention of EMNIST character images.
- 5. The resulting 28x28 image is normalized to $[0, 1]$ and reshaped with a channel and batch dimension, exactly mirroring the preprocessing used during training.
- 6. The CNN produces a probability distribution over the 62 classes; the class with the highest probability is returned as the predicted character, and its probability is reported as a confidence percentage.
- 7. The top 5 highest-probability classes and their associated percentages are also computed and displayed as alternative predictions.

[INSERT SCREENSHOT HERE — Figure 7: Final Handwritten Character Recognition UI]

Figure 7: Final Handwritten Character Recognition UI

17. User Interface

The prediction interface was built with Gradio and features a dark, gradient-themed layout. The interface actually implements the following features, as defined in the notebook:

- Image upload — a dedicated image input component for uploading a handwritten character photo or scan.
- A "Recognize Character" button that triggers the prediction pipeline described in Section 16.
- A "Clear" button that resets the uploaded image.
- Predicted character display — a read-only text box showing the single most likely predicted character.
- Confidence score display — a read-only text box showing the confidence percentage of the top prediction.
- Top 5 predictions display — a multi-line text box listing the five highest-probability classes with their individual confidence percentages.
- Model performance information — a summary panel stating the model type (Custom CNN), dataset (EMNIST ByClass), number of classes (62), and test accuracy (87.04%), plus header "stat" cards repeating the test accuracy, macro F1-score (74.18%), class count, and model type.
- A "How It Works" explanatory panel outlining the four-step pipeline: Upload Image -> Preprocess -> CNN Prediction -> Character Result.

The interface was launched with public link sharing enabled (`demo.launch(share = True, debug = True)`), producing a temporary public Gradio URL for external access during the Colab session.

18. Applications

Handwritten character recognition systems of this kind have a broad range of real-world applications, including:

- Optical Character Recognition (OCR) pipelines for digitizing handwritten documents.
- Automated digit recognition for forms, cheques, and numeric entry fields.
- Form processing and data entry automation in administrative or financial workflows.
- Educational tools, such as handwriting practice and automated grading aids.
- Document digitization and archival of handwritten records.
- Postal address and numeric code recognition.
- Handwriting-based human-computer interfaces, such as stylus or touchscreen character input.

19. Limitations

- EMNIST ByClass consists of isolated, pre-segmented single characters rather than complete handwritten words or sentences, so the model cannot directly recognize connected or cursive handwriting without an additional segmentation step.
- The dataset exhibits significant class imbalance between digits and letters, and among individual letters (Table 1), which likely contributes to weaker performance on the least-represented classes.
- Certain characters are visually near-identical in isolated form (e.g., lowercase/uppercase pairs such as c/C, o/O, s/S, u/U), which is an intrinsic source of ambiguity that is difficult to resolve from a single character image without additional context.
- Performance in the live prediction interface depends heavily on image quality, lighting, and how well the automatic background-detection and cropping logic isolates the character; poorly centered, noisy, or low-contrast uploads may be preprocessed incorrectly.
- There is a substantial gap between class-wise performance, with several lowercase letters achieving F1-scores below 0.15 while many digits exceed 0.95, meaning overall accuracy alone can understate weaknesses in specific classes.
- The data augmentation pipeline that was implemented was not actually applied during the training of the final model, so its potential regularization benefit was not realized in the reported results.

20. Future Improvements

- Actually incorporate the implemented data augmentation layers into the training pipeline (e.g., as part of the model or via an augmented `tf.data` pipeline) rather than only visualizing them.
- Apply the already-computed class weights (or alternative class-imbalance techniques such as oversampling) to the CNN training run to address the weak performance on minority and visually ambiguous classes.
- Explore more advanced CNN architectures (deeper networks, batch normalization, residual connections) to improve accuracy on the weaker classes identified in Section 13.2.
- Revisit transfer learning approaches (such as the EfficientNetB0-based pipeline experimented with in the notebook) with a longer, properly tuned training schedule, since the version explored in this project did not reach a usable accuracy within the training time available.
- Introduce more sophisticated, task-aware augmentation (e.g., elastic distortions) known to be effective for handwriting recognition.
- Extend the system with CRNN/sequence-based models capable of recognizing full handwritten words or sentences instead of isolated single characters.
- Investigate real-time handwriting recognition, e.g., recognizing characters as they are drawn stroke-by-stroke.
- Package the model and interface for mobile or web deployment beyond the temporary Gradio share link used during development.

21. Conclusion

This project developed a handwritten character recognition system for the EMNIST ByClass dataset, covering all 62 digit and letter classes. A compact CNN with two convolutional blocks and a dense classification head was trained on 628,138 preprocessed, normalized training images and validated on a stratified 69,794-image split. On the fully held-out test set of 116,323 images, the model achieved a test accuracy of approximately 87.04% and a test loss of 0.3537, with a macro-average F1-score of 74.18% and a weighted-average F1-score of 85.65%.

The per-class evaluation showed that the model performs strongly on digits and visually distinctive letters, while struggling with a specific subset of lowercase letters that closely resemble their uppercase counterparts — a limitation consistent with the intrinsic ambiguity of the EMNIST ByClass label space rather than a flaw unique to this architecture. The trained model was successfully integrated into an interactive Gradio-based prediction interface that preprocesses uploaded handwriting images and returns a predicted character with a confidence score and top-5 alternatives. Overall, the project demonstrates a functional, end-to-end handwritten character recognition pipeline, while also identifying concrete, evidence-based directions — applying the already-implemented augmentation and class-weighting techniques to training — for improving performance in future iterations.

22. Technologies and Tools

The following tools and libraries were actually used in the notebook:

- Python — primary programming language
- Google Colab — development and execution environment (indicated by `/content/` file paths and Colab-specific runtime messages)
- TensorFlow (version 2.20.0) and Keras (`tf.keras`) — model construction, compilation, and training
- NumPy — numerical array processing
- Matplotlib — visualization of sample images and augmentation results
- scikit-learn — train/validation splitting (`train_test_split`), `classification_report`, and `compute_class_weight`
- Pillow (PIL) — image loading and preprocessing in the prediction pipeline
- psutil — runtime RAM monitoring
- Gradio — the interactive web-based prediction interface
- `gzip / struct / urllib` (Python standard library) — downloading and parsing the raw EMNIST IDX dataset files

`tensorflow_datasets` was installed and imported in the notebook but the dataset was ultimately loaded via a manual IDX-file download and parser rather than through this library. [Not available in the notebook]: any Pandas usage was not present in the reviewed cells.

23. References

Cohen, G., Afshar, S., Tapson, J., & van Schaik, A. (2017). EMNIST: an extension of MNIST to handwritten letters. arXiv preprint arXiv:1702.05373.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Chollet, F., et al. Keras documentation. <https://keras.io/>

Abadi, M., et al. TensorFlow: Large-scale machine learning on heterogeneous systems. <https://www.tensorflow.org/>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Abid, A., et al. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. <https://gradio.app/>

Hugging Face Datasets Hub — "Roey30ne/emnist-byclass" dataset mirror, used as the data source in this project.

24. Appendix

Appendix A — Full Per-Class Classification Report (Test Set)

Class	Precision	Recall	F1-score	Support
0	0.6850	0.8392	0.7543	5,778
1	0.6809	0.9191	0.7823	6,330
2	0.9591	0.9782	0.9685	5,869
3	0.9920	0.9945	0.9932	5,969
4	0.9695	0.9779	0.9737	5,619
5	0.9531	0.9403	0.9467	5,190
6	0.9744	0.9818	0.9781	5,705
7	0.9872	0.9943	0.9907	6,139
8	0.9763	0.9863	0.9813	5,633
9	0.9327	0.9768	0.9542	5,686
A	0.9445	0.9774	0.9607	1,062
B	0.9353	0.9367	0.9360	648
C	0.7785	0.9540	0.8574	1,739
D	0.8860	0.9076	0.8966	779
E	0.9693	0.9647	0.9670	851
F	0.7715	0.9611	0.8559	1,440
G	0.9297	0.8881	0.9085	447
H	0.9455	0.9328	0.9391	521
I	0.6608	0.4795	0.5557	2,048
J	0.8518	0.9089	0.8794	626
K	0.6199	0.8665	0.7227	382
L	0.9135	0.9383	0.9257	810
M	0.7691	0.9468	0.8488	1,485
N	0.9302	0.9867	0.9576	1,351
O	0.6706	0.5320	0.5933	4,156
P	0.8326	0.9435	0.8846	1,397
Q	0.8892	0.9128	0.9008	413
R	0.9453	0.9604	0.9528	809
S	0.8250	0.9396	0.8786	3,508
T	0.9313	0.9461	0.9386	1,576
U	0.7737	0.9510	0.8532	2,002

Class	Precision	Recall	F1-score	Support
V	0.7374	0.5151	0.6065	796
W	0.7966	0.8598	0.8270	806
X	0.7426	0.7014	0.7214	432
Y	0.7376	0.8559	0.7923	798
Z	0.7411	0.6293	0.6807	464
a	0.9395	0.9075	0.9233	1,644
b	0.8926	0.8769	0.8847	853
c	0.3478	0.0185	0.0352	432
d	0.9851	0.9798	0.9824	1,683
e	0.9781	0.9839	0.9810	4,092
f	0.3333	0.0250	0.0465	400
g	0.7335	0.5654	0.6385	589
h	0.9545	0.9506	0.9526	1,479
i	0.8132	0.3466	0.4860	427
j	0.8063	0.6435	0.7158	317
k	0.8333	0.5472	0.6606	466
l	0.5385	0.2371	0.3292	2,535
m	0.4729	0.1315	0.2057	464
n	0.9603	0.9420	0.9511	1,898
o	0.5000	0.0064	0.0127	466
p	0.6421	0.3315	0.4373	368
q	0.6966	0.4000	0.5082	505
r	0.9721	0.9621	0.9671	2,320
s	0.7500	0.0069	0.0136	437
t	0.9560	0.9454	0.9507	2,965
u	0.4839	0.0622	0.1103	482
v	0.4674	0.6902	0.5574	468
w	0.7367	0.6231	0.6752	467
x	0.7228	0.7489	0.7356	470
y	0.6667	0.3307	0.4421	381
z	0.6617	0.5898	0.6237	451

Table 5: Full per-class precision, recall, F1-score, and support on the 116,323-image test set (digits=63,240 total support across classes 0-9; letters=53,083 total support across classes A-Z, a-z).

Aggregate	Precision	Recall	F1-score	Support
Accuracy	-	-	0.8704	116,323
Macro Average	0.7981	0.7393	0.7418	116,323
Weighted Average	0.8622	0.8704	0.8565	116,323

Appendix B — Key Code Snippets

Final CNN model definition (as it appears in the notebook):

```
cnn_model_1 = models.Sequential([ layers.Input(shape=(28, 28, 1)), layers.Conv2D(32,
(3, 3), activation="relu", padding="same"), layers.MaxPooling2D((2, 2)),
layers.Conv2D(64, (3, 3), activation="relu", padding="same"), layers.MaxPooling2D((2, 2)),
layers.Flatten(), layers.Dense(128, activation="relu"), layers.Dropout(0.30),
layers.Dense(62, activation="softmax") ])
```

Model compilation and training call:

```
cnn_model_1.compile( optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
loss="sparse_categorical_crossentropy", metrics=["accuracy"] ) history_cnn_1 =
cnn_model_1.fit( X_train, y_train, validation_data=(X_val, y_val), epochs=15,
batch_size=256, callbacks=[early_stopping, checkpoint, reduce_lr], verbose=1 )
```

Appendix C — Additional Screenshots and Prediction Examples

[INSERT SCREENSHOT HERE — EMNIST Dataset Sample Characters]

Figure 1: EMNIST Dataset Sample Characters

[INSERT SCREENSHOT HERE — Additional prediction examples from the live Gradio interface]